

XRPBlocks: updates through 20 September 2026

This is the current addendum to changes 1-18 in the replication guide. It records the final implementation, including corrections made during testing. Earlier PDF pages are retained as historical documentation; this addendum takes precedence where behavior differs.

19. Onboard RGB colour and brightness

The Board category adds a named-colour dropdown with brightness from 0 to 100%. Eleven choices match the NeoPixel palette, including off. Default brightness is 30%. Generated Python clamps brightness and scales RGB values. It requires no NeoPixel add-on library. English and Dutch labels are available. The original three-channel RGB block remains available.

20. Delete unused blocks

Right-click the workspace or a block and choose Delete unused blocks. Cleanup removes disconnected stacks and unreachable functions. It preserves start blocks, explicitly protected blocks, and functions called directly or indirectly by retained code, including recursive calls. Default shadow inputs do not incorrectly protect a disconnected stack. Deletion is grouped for Undo. The option is disabled when there is nothing eligible to remove.

21. Bottom panel and variable cleanup

The whole Python/Console panel starts collapsed unless pinned. Both tabs offer a shared pin control, remembered in the browser. Unpinning hides the panel. Unpinned tabs open manually; Run and Deploy do not force them open. Clicking elsewhere no longer dismisses a manually opened panel. Resizing preserves the expanded size without overriding the collapsed height.

Variables with no references anywhere in the workspace are removed on app startup and before Run. References in loose blocks and procedure parameters are retained. Cleanup is grouped for Undo and the workspace is saved.

22. Remembered connection and uninterrupted startup

Connect first tries the last successful authorised USB or Bluetooth device. USB uses the exact in-session port when possible; after reload it matches USB vendor/product IDs only when a single authorised port matches. Missing, ambiguous or failed connections return to the chooser. Bluetooth reuse depends on browser support for previously authorised devices. Repeated Connect clicks are guarded while connection work is in progress.

Connect no longer sends a stop/REPL-recovery command. This preserves a web server or other main.py program already running on power-up. Stop remains available whenever connected. Run, Stop and Deploy can interrupt execution. Deploy saves main.py but does not run it: click Run or restart the board.

23. Mecanum wheels library, version 1.1.0

Four encoded motors drive standard X-pattern mecanum wheels. Default ports are 1 front-left, 2 front-right, 3 rear-left and 4 rear-right. The palette provides two compact setup blocks: motor assignment and chassis geometry. The older combined setup block remains registered for saved-project compatibility.

Controls include eight directions, arbitrary movement angle, continuous drive, distance in centimetres, turning in degrees, Stop, wheel reversal and a last-move-completed Boolean. Speed is 0-100% of configured maximum

wheel RPM, not a guarantee of chassis speed. Distance and turns use encoder estimates, with timeouts and stopping on completion/interruption. Wheel diameter and wheelbase/track defaults are placeholders that must be measured and calibrated. Mecanum hardware operation has not been verified in this session.

24. Remote control library

Remote control is independent of motors and mecanum. Its MicroPython driver hosts an HTML page on port 80, either through a robot hotspot or by joining an existing network. The page has six customizable hold buttons, local label editing, connection status and Release all buttons. Blocks read held state, one-shot press events, connection status and page address, and configure labels, title, networking, updates and shutdown.

An active controller sends heartbeat messages. Button reads become false after one second without messages. This does not stop actuators by itself: user code must handle the false/disconnected branch with Stop or Off. Only one controller owns the inputs at a time; sequence numbers reject stale messages. The web service is cooperative, not a background thread. Button/connection reads also service it automatically; use remote update in loops without those reads. Avoid long blocking commands while controlling devices live.

25. Remote page layout choice

Choose Button grid or Game D-pad on the page. The browser remembers the layout and custom labels. Button mapping stays fixed: Up=1, Left=2, Right=3, Down=4, A=5, B=6. Switching layout releases held states. Symbols do not assign any motor behavior. Space/Enter operate a focused button; mouse and touch are supported.

26. Remote networking fixes, current driver 1.1.1

The XRP CYW43 hotspot uses WPA2 AES authentication flags (0x400004), replacing an incorrect generic enum value. A previous RemoteControl instance is closed before rebinding port 80 on a subsequent run. Startup prints its driver version, network status and the actual HTTP address.

Join network resets the old station association, supports an empty password for open networks, reports progress every five seconds, and has a 30-second timeout. It distinguishes a rejected password, missing network and general connection failure. Use 2.4 GHz Wi-Fi; a router-assigned address replaces the hotspot address. Captive portal and enterprise login are not implemented.

Latest hardware evidence: the updated driver ran but reported that network Fibre was not found. Join mode is therefore not confirmed working on that network. The user previously reported the remote/upload setup working before testing join mode. This is not evidence of complete hardware validation.

27. Confirmed USB file uploads and module refresh

USB uploads now write UTF-8 data in 256-byte chunks, represented as hex, with a raw-REPL acknowledgement after each command. Data goes to a temporary .upload file. The byte count is checked before renaming it over the destination. Failed driver installation aborts Run/Deploy instead of silently continuing.

After a driver upload, the USB path removes its cached module from sys.modules so the next import loads the new file. RemoteControl's previous listener is closed first. main.py is not treated as an imported driver. This corrected logs where 1.1.1 had uploaded but cached 1.1.0 still ran. Upload acknowledgements remain visible in Console. Bluetooth retains its existing separate binary uploader; these confirmed-write/module-refresh changes apply to USB.

28. OLED scrolling text, library version 2.1.0

New block: OLED scroll [message] on line [1] speed (pixels/sec) [30]. Place it inside a forever loop with a short wait, for example 0.02 seconds. Long text scrolls left, pauses at the ends and repeats. Text that fits stays still. Speed is clamped to 1-120 pixels/second; each line has independent state. The method returns immediately, allowing other loop work to continue. In manual display-update mode, also call OLED update. Static line replacement or Clear resets the corresponding scrolling state. Hardware scrolling remains unverified.

Verification and outstanding work

Current committed tests: tools/test_mecanum.py (7 simulated tests) and tools/test_remote_control.py (8 protocol/network tests). Driver embedding is checked by tools/embed_drivers.py --check. Additional session checks covered Blockly registration and Python generation, all library flyouts, phone-width page rendering, button press/release, D-pad mapping and persistence, real local HTTP requests through the Remote control driver, simulated USB transfers/errors, and OLED scrolling with a stub framebuffer. These do not replace XRP hardware tests.

The previous guide's 913 checks are historical reported results, not a newly rerun suite. The older harnesses are not present in this checkout. Outstanding: join-network testing on a visible 2.4 GHz network, mecanum calibration and hardware validation, OLED scrolling on the display, and Bluetooth driver-update parity. The six legacy expander/NeoPixel files listed in section 3.11 of the guide remain present but unused; they have not been deleted.

Reproduce and use

Use the refreshed source snapshots in XRPBlocks-replication-guide.md together with its new additional-files section. Run python tools/embed_drivers.py, then python tools/embed_drivers.py --check. Start locally with start-xrpblocks.bat or python serve.py; do not open index.html through file://.

For a library update: refresh the IDE, connect USB, then Run or Deploy to upload the new driver. Deploy alone leaves the program stopped. For standalone use, restart after deploying main.py. New libraries are added through Library.

Detailed usage: docs/mecanum.md and docs/remote-control.md. The OLED scroll block tooltip explains loop placement and manual-update behavior.

Historical documentation

The following 22 pages preserve the original local-modification report and its earlier addendum. Dates, change counts, test totals and implementation details in those pages describe earlier versions. The 20 September 2026 update preceding this divider takes precedence.

XRPBlocks: Local Modifications

Changes made to the local copy of the XRPBlocks block-programming IDE for the XRP robot

Prepared for: Edward Gentle, Subject Advisor CAT & IT, Capricorn South District, Limpopo Department of Education

Date: 18 September 2026 (revised)

Installation: C:\Users\Gentle\Desktop\XRP\XRPBlocks-main\XRPBlocks-main

Upstream project: Stichting STEAMup / XRPBlocks (MIT licence) — github.com/Stichting-STEAMup/XRPBlocks

1. Summary

Twelve changes were made to a local, unmodified download of XRPBlocks: six fix defects that prevent the IDE or the generated programs from working, and six add capability the toolbox did not have. Section 12 lists every file touched. A companion document, `XRPBlocks-replication-guide.md`, carries the full source of every changed file and reproduces the whole installation from a clean clone.

#	Change	Problem it solves	Type
1	Local web-server launcher (<code>start-xrpblocks.bat</code>)	Opening <code>index.html</code> from disk, as the project README instructs, silently breaks the app: Connect XRP does nothing	Deployment fix
2	New repeat forever block	The Loops category had no infinite-loop block, which is the first loop most beginners need	Feature
3	Shadow value on set variable to	Blockly builds that block with an empty socket, so beginners read it as broken or missing	Usability fix
4	MicroPython compatibility patch	Any program using change variable by fails on the robot with <code>ImportError</code> before a single line runs	Bug fix (critical)
5	PCF8575 I/O expander driver and block category	No way to drive extra LEDs, buttons, relays or buzzers from the Qwiic connector	Feature
6	NeoPixel strip driver and block category	No way to control an addressable LED strip	Feature
7	Confirmation dialog before Deploy	Deploy silently overwrote the program saved on the robot	Safety
8	Servo category rebuilt	Only two of four servos offered; angles were not range-checked; the documented range was wrong	Feature and safety
9	Motors category rebuilt	Untranslated labels, effort not range-checked, tooltips with no units	Feature and safety
10	Gyro reset and heading blocks	Yaw accumulates from power-up and no block could zero it, so gyro programs appeared frozen	Bug fix
11	Dropdowns replacing repeated blocks	Eight near-identical blocks where three would do	Usability
12	Toolbox tolerates blocks the browser has not loaded	One stale cached file broke every category, not just its own	Robustness

Changes 5 and 6 each add a MicroPython driver that is not part of XRPLib. The IDE installs these onto the robot's filesystem by itself, the first time a program needs one, so there is no separate step in XRPCode or Thonny.

2. Change 1: Running XRPBlocks locally

Problem

The upstream README says to open `index.html` in Chrome or Edge, with no build step. That does not work. The page loads its own application code as an ES module:

```
<script src="js/vendor/blockly/blockly_compressed.js"></script>    <!-- plain script, loads -->
<script type="module" src="js/app.js"></script>                  <!-- module, BLOCKED -->
```

Chrome and Edge block module scripts loaded from a `file://` origin under CORS (origin `null`). The Blockly canvas draws,

because those vendor files are plain scripts, but the application code that wires up the Connect XRP button never runs. The symptom is a page that looks correct and does nothing.

Solution

Serve the folder over `http://localhost` , which is a secure context and therefore permits both module scripts and the Web Serial API. A launcher was added to the installation folder:

File added	<code>start-xrpblocks.bat</code> NEW
Port	8765 (checked for availability before starting)
Binding	<code>127.0.0.1</code> only, so no Windows Firewall prompt appears
Server	Tries <code>py</code> , then <code>python</code> , then <code>npx serve</code> ; prints installation guidance if none is present
Behaviour	Prints the folder being served, opens the default browser after two seconds, stays open until closed

Lab deployment note. Web Serial requires a secure context, meaning `localhost` or HTTPS. XRPBlocks therefore cannot be served from one school machine to the rest over plain `http://192.168.x.x` : the page will load but the robot will never connect. Each machine needs its own copy of the folder and its own local server, unless HTTPS with a certificate is set up.

3. Change 2: The repeat forever block

Problem

The Loops category shipped with *repeat n times*, *repeat while/until*, *for* and *for each*, but no forever block. In Scratch-style teaching the forever loop is the first loop learners meet, and every sensor-driven robot program needs one.

Solution

A new `xrp_forever` block was added at the top of the Loops category, styled with the existing `loop_blocks` theme entry so it matches the other loop blocks.

The block deliberately has **no bottom connector**. Nothing can be attached after it, which makes the idea that the program never continues past this point visible in the shape of the block itself.

Generated Python, with `pass` substituted when the loop is still empty so that a half-built program never produces broken code:

```
while True:
    time.sleep(0.5)
    board.set_rgb_led(Red, Green, Blue)
```

Files added	<code>js/blockly/blocks/loops.js</code> NEW <code>js/blockly/generators/loops.js</code> NEW
Files modified	<code>js/app.js</code> MODIFIED (import and register) <code>js/blockly/toolbox.js</code> MODIFIED (Loops category) <code>js/ui/translations.js</code> MODIFIED (en and nl strings)

4. Change 3: Shadow value on "set variable to"

Problem

Blockly builds the Variables flyout itself, and it is inconsistent: *change item by* is created with a shadow number already plugged in, while *set item to* is created with an **empty** socket. Beginners read the empty socket as a broken block, and conclude that there is no way to assign a value.

Solution

`js/app.js` now wraps Blockly's built-in Variables category callback and inserts a shadow `math_number` of 0 into the `VALUE` socket, so the block arrives ready to type into, exactly like the Board and Drive blocks. The wrapper is inside a try/catch: if a future Blockly version changes the API, the flyout falls back to the original behaviour rather than breaking the page.

```
_customiseVariablesFlyout() {
  try {
    this.workspace.registerToolboxCategoryCallback('VARIABLE', (ws) => {
      const items = Blockly.Variables.flyoutCategory(ws, false);
      for (const item of items) {
        if (item && item.kind === 'block' && item.type === 'variables_set' && !item.inputs) {
          item.inputs = {
            VALUE: { shadow: { type: 'math_number', fields: { NUM: 0 } } },
          };
        }
      }
      return items;
    });
  } catch (err) {
    console.warn('Could not customise the Variables flyout; using the Blockly default.', err);
  }
}
```

Reminder for classroom use: the set, change and get blocks appear only after at least one variable has been created with the **Create variable...** button. Before that the category shows only the button.

5. Change 4: MicroPython compatibility (critical)

Problem

Blockly's Python generator targets desktop CPython. The **change variable by** block emits a type-guarded increment that depends on the CPython standard-library module `numbers`:

```
import time
from numbers import Number          # <-- MicroPython has no such module
...
Blue = (Blue if isinstance(Blue, Number) else 0) + 1
```

MicroPython does not ship a `numbers` module, so the robot raises `ImportError: no module named 'numbers'` on the second line and the entire program stops before executing anything. The observed symptom was an LED program that did nothing at all, not even the colour set before the loop, while an otherwise identical program using random values worked perfectly. The only difference between the two was the presence of a **change variable by** block.

This was confirmed by running the XRPBlocks generator against the failing program outside the browser and inspecting the generated Python directly.

Three further standard blocks emit the same import: *is prime* (`math_number_property`) and the *average* and *median* options of the list-maths block.

Solution

A new module, `js/blockly/generators/micropython-compat.js`, applies two fixes:

1. **Regenerates change variable by** as a plain addition, with no import and no type check.
2. **Substitutes a shim** for any remaining request for `numbers`. Because `isinstance()` accepts a tuple, `Number = (int, float)` behaves correctly for every generated check.

Verified result

The same program now generates code that runs on the robot:

```
import time

Blue = None
Green = None
Red = None

board.set_rgb_led(60, 0, 0)
time.sleep(3)
Blue = 20
Green = 20
Red = 20
board.led_on()
while True:
    time.sleep(0.5)
    Blue = Blue + 1
    board.set_rgb_led(Red, Green, Blue)
```

Worth reporting upstream. This defect breaks every XRPBlocks program that uses the standard *change variable by* block, which includes most counting and sensor-loop examples. It is a candidate for a GitHub issue or pull request to Stichting STEAMup.

6. Change 5: PCF8575 I/O expander

The PCF8575 is a 16-bit I/O expander on the I2C bus, giving sixteen extra pins from a Qwiic connector. A new **PCF8575** category drives it, and `lib/PCF8575.py` is the MicroPython driver behind those blocks.

The wiring rule that governs the design

The PCF8575 has quasi-bidirectional pins: there is no direction register. A pin is an input simply by being left HIGH, which the chip does with a weak current source of **30 to 300 uA**. A pin driven LOW sinks a proper **10 to 25 mA**. Everything is therefore wired active low:

Device	Wiring
LED	Anode to VDD through a resistor, cathode to the expander pin. The pin sinks the current, so driving it LOW lights the LED. Wired the other way round it barely glows.
Button	Switch between the pin and GND. The chip's own weak pull-up holds it HIGH when released. No external resistor needed.

The driver keeps a shadow register, so setting one pin never disturbs the others, and any pin declared as a button is always written HIGH, which is what makes it readable at all. Pins are numbered 0 to 15, where 0 to 7 are P00 to P07 (the first data byte) and 8 to 15 are P10 to P17 (the second), matching the datasheet's byte order. Addresses are validated against the 0x20 to 0x27 range set by the A2 A1 A0 pins. If no bus is supplied, the driver scans Qwiic 1 (I2C1, which the IMU also uses) then Qwiic 0 (I2C0), by board pin name with a GPIO fallback, so it works on either connector.

The blocks

- set up PCF8575 at address `0x20`
- use PCF8575 pin `N` as LED (output) or button (input)
- PCF8575 LED on pin `N` on / off, and switch all PCF8575 LEDs off
- PCF8575 button on pin `N` is pressed (a boolean, drops straight into an if)
- set PCF8575 pin `N` to high / low, and PCF8575 pin `N` level, for relays and buzzers

```
from PCF8575 import PCF8575, INPUT, OUTPUT

expander = PCF8575(address=0x21)

expander.pin_mode(0, OUTPUT)
expander.pin_mode(8, INPUT)
expander.all_off()
while True:
    if expander.button(8):
        expander.led(0, True)
```

The category was first labelled "Expander" and was renamed to **PCF8575** so the blocks name the actual chip. Only the display labels changed, in English and Dutch; the block identifiers are untouched, so saved workspaces still load.

7. Change 6: NeoPixel strip

A new **NeoPixel** category drives a WS2812 strip, backed by `lib/NeoPixelStrip.py`.

A NeoPixel strip cannot run off the PCF8575. WS2812 pixels take a single-wire stream at 800 kHz, where each bit lasts 1.25 us. A PCF8575 pin only changes when an I2C transaction pushes it, on a bus running at 400 kHz at best, so each change costs tens of microseconds: roughly two orders of magnitude too slow. The pin also cannot source current, only sink it, so the rising edges would be far too slow. The data line must go to a real GPIO.

Where to connect it

The pin dropdown offers the four servo headers, which are the easiest physical connector, and the eight pins SparkFun's pinout lists as free:

Servo headers	Free GPIO
Servo 1 = GPIO 6, Servo 2 = GPIO 9, Servo 3 = GPIO 7, Servo 4 = GPIO 8	GPIO 12 to GPIO 19

The blocks

- set up NeoPixel strip on `pin` with `N` pixels
- set NeoPixel brightness to `N` per cent
- colour `[named]`, and colour red `R` green `G` blue `B`, as reporter blocks that plug into the action blocks
- set NeoPixel `N` to a colour, set the whole strip to a colour, switch the strip off
- rainbow across the strip shifted by `N`, and shift the strip by `N`
- show NeoPixel changes, update the strip straight away or only on show, and number of NeoPixels

By default every change appears at once, so a single block does something visible. Switching updating to "only when I say show" turns the strip into a proper display buffer, which is the teaching point worth making: nothing reaches the strip until **show**.

Brightness is a percentage applied when writing, so the colours set are remembered at full value and turning the brightness back up restores them exactly. Pixels are numbered from 0, matching the Python list underneath. Channel values are clamped, so a stray 300 or a negative number cannot crash the write.

```
from NeoPixelStrip import NeoPixelStrip, colour
import time

strip = NeoPixelStrip(pin=6, count=10)

strip.set_brightness(25)
strip.set_auto_show(False)
strip.fill(colour('blue'))
strip.set_pixel(0, (255, 120, 0))
strip.show()
Step = 0
while True:
    strip.rainbow(Step)
    Step = Step + 8
    strip.show()
    time.sleep(0.05)
```

Three hardware points before class

Point	Detail
Power	Ten pixels at full white draw roughly 600 mA, far more than the board should supply. For more than two or three pixels give the strip its own supply and tie the grounds together. The 25 per cent default brightness keeps the draw sensible.
Logic level	A WS2812 on 5 V wants a data HIGH of about 3.5 V and the XRP gives 3.3 V. It often works and is often flaky. Running the strip at about 3.7 to 4.2 V, or adding a level shifter, is the reliable fix.
Servo header	Convenient as a connector, but check what voltage it actually supplies before powering pixels from it.

8. Change 7: Confirmation before saving to the robot

XRPBlocks already had a **Deploy** button, which writes the program to the robot as `main.py`. That filename is required rather than conventional: MicroPython runs `boot.py` and then `main.py` from the filesystem root at startup, so a program saved under any other name never starts by itself. Deploy overwrote whatever was already there without asking.

A confirmation dialog was added, in the same style as the connection picker:

Title	Save this program to the robot?
Body	The program will be saved on the XRP as <code>main.py</code> and will run by itself every time the robot is switched on.
Warning	Any program already saved on the robot will be replaced.
Buttons	Cancel, Save to XRP

Two deliberate safety choices: **Cancel takes the focus** when the dialog opens, and there is **no Enter shortcut**, so a stray keypress cannot overwrite a learner's robot. The close button, the Escape key and a click on the backdrop all resolve to cancel. The new module, `js/ui/confirm-modal.js`, is reusable for anything else destructive.

9. Changes 8 and 9: Servos and motors

Only half the hardware was reachable

The Servo category offered servos 1 and 2 although the XRP Controller has four headers, and XRPLib's `defaults.py` exports `servo_three` and `servo_four` when the board has those pins. The Motors category did list four motors, but the labels for motors 3 and 4 were hardcoded English while left and right used translation keys, so the Dutch interface read "links, rechts, motor 3, motor 4".

Neither library call checks its range

Confirmed by reading the XRPLib source:

Call	Behaviour
<code>Servo.set_angle(degrees)</code>	Takes 0 to 200 , which spans the full 500 to 2500 microsecond pulse, so 100 is the centre, not 90 . No clamping: the value is converted straight to a pulse width, so a learner typing 500 drives the servo hard against its stop. The old tooltip said "0 to 180", which was wrong, and the toolbox default was 90, slightly off centre.
<code>EncodedMotor.set_effort(effort)</code>	Takes -1 to 1 , passed straight to the motor driver. No clamping. Learners who think in percent and type 50 get full effort with no warning.
<code>EncodedMotor.set_speed(rpm)</code>	Units are rpm . Passing <code>None</code> or <code>0</code> turns speed control off and hands the motor back to the effort calls; it does not brake.

Every block now routes through a small clamping helper that the generator emits once per program, for example:

```
def xrp_servo_angle(servo, degrees):
    if degrees < 0:
        degrees = 0
    elif degrees > 200:
        degrees = 200
    servo.set_angle(degrees)
```

The two categories now

Servo, five blocks	Motors, ten blocks
set servo N angle to 100 degrees centre servo N sweep servo N from 0 to 200 over 1 seconds set servo N continuous speed to 50 % free servo N	set motor N effort 0.5 set motor N effort 50 % set motor N speed 60 rpm turn speed control off for motor N motor N position in turns / in counts / speed in rpm reset motor N encoder, brake, coast

Sweep moves the servo smoothly at 50 steps a second and blocks until it finishes. The continuous-speed block maps -100 to 100 onto the servo's 0 to 200 range by simple addition, which suits a continuous rotation servo whose stop point is the centre pulse; a standard servo just moves to an angle instead. All tooltips were rewritten with units.

10. Change 10: Why gyro programs appeared frozen

Found on hardware. A program that pointed a servo at the robot's heading drove the servo hard to one end and stayed there. Reading XRPLib's `imu.py` explained it:

- `get_yaw()` simply returns `self.running_yaw`; it reads nothing itself.
- `running_yaw` is accumulated by a background `Timer` callback, `_update_imu_readings()`, which starts when the IMU object is created.
- `calibrate()` recomputes the gyro offsets and restarts that timer, but **never zeroes `running_yaw`**.

So yaw had been accumulating, with uncalibrated offsets, since the board powered up. By the time the program ran it was already past the servo's range, and the clamp pinned the servo at its end stop. There was no block to zero it.

Added: set gyro [yaw / pitch / roll] to zero, which calls `imu.reset_yaw()`, `reset_pitch()` or `reset_roll()`, and gyro heading (0 to 360), the wrapped reading, since `get_yaw()` is unbounded and keeps counting past 360. The Sensors category was reordered to read in the order it is used: calibrate, zero, then the readings.

The working program, confirmed on the robot:

```
imu.calibrate()
imu.reset_yaw()
while True:
    xrp_servo_angle(servo_four, 100 + imu.get_yaw())
    time.sleep(0.02)
```

The `+ 100` centres the signed yaw in the servo's range. The `sleep` matters as well: without it the loop starves the IMU's background timer and floods the bus.

11. Changes 11 and 12: Dropdowns, and surviving a stale cache

Eight blocks became three

Was	Now
gyro yaw / heading / pitch / roll, four blocks	gyro [yaw ▼], four readings
line sensor left / right, two blocks	line sensor [left ▼] reflectance
left / right encoder, two blocks	[left ▼] wheel position

The retired block types remain **registered but hidden from the toolbox**, so workspaces saved before the change still load and still generate correct code.

The cache failure, and the two fixes

Most of the debugging time in this project went to a single failure mode. Chrome caches ES modules aggressively, so after an edit the browser can run a **new `toolbox.js` against an old block file**. The toolbox then asks for a block the browser has never heard of, and Blockly raises:

```
Uncaught TypeError: Invalid block definition for type: xrp_motor_set_effort_percent
    at BlockFlyoutInflater.createBlock ...
Uncaught Error: Block not present in workspace's list of top-most blocks.
```

That breaks the category's flyout **and every category opened afterwards**, which is why the symptom appeared to move from one category to the next as each change landed. Two fixes:

serve.py	A small static server that sends <code>Cache-Control: no-store</code> on everything, so a normal refresh always loads the latest edit. The launcher now uses it, and it prints each request and the folder it is serving, which makes a wrong-folder mistake obvious.
Toolbox pruning	Before building the flyouts, the app checks every block the toolbox names against what the browser actually loaded, hides any it cannot build, and reports them in the console and as a toast. The worst case becomes a missing block with an explanation, never a dead toolbox.

12. Known defects not yet fixed

Item	Detail
blink LED block	The block generates <code>board.led_blink(count, delay)</code> , but the XRPLib method signature is <code>led_blink(frequency)</code> , a single argument. This will almost certainly raise a <code>TypeError</code> on the robot. Not yet confirmed on hardware. Avoid this block in class until tested.
RGB values above 255	XRPLib's <code>set_rgb_led()</code> performs no range checking; it passes the tuple straight to the NeoPixel, which stores bytes. A counting loop that pushes a channel past 255 stops with a <code>ValueError</code> . Teach the <i>remainder of block</i> (<code>value % 256</code>) to wrap the value. This makes a useful lesson on why a byte holds 0 to 255.
XRP Beta boards	<code>set_rgb_led()</code> raises <code>NotImplementedError</code> on the XRP Beta. Only the newer XRP Controller has the onboard NeoPixel.
Upstream issue #13	The Web Serial device chooser does not filter to XRP devices, so learners see every serial device on the machine. Open in the upstream project.

13. File manifest

All paths are relative to `C:\Users\Gentle\Desktop\XRP\XRPBlocks-main\XRPBlocks-main`.

File	Status	Purpose
<code>start-xrpblocks.bat</code>	NEW	Local web server launcher (change 1)
<code>js/blockly/blocks/loops.js</code>	NEW	Definition of the repeat forever block
<code>js/blockly/generators/loops.js</code>	NEW	Python generator for repeat forever
<code>js/blockly/generators/micropython-compat.js</code>	NEW	MicroPython compatibility fixes (change 4)
<code>lib/PCF8575.py</code>	NEW	MicroPython driver for the I/O expander (change 5)
<code>js/lib/pcf8575-source.js</code>	NEW	The same driver embedded for automatic installation on the robot
<code>js/blockly/blocks/expander.js</code>	NEW	PCF8575 block definitions
<code>js/blockly/generators/expander.js</code>	NEW	Python generators for the PCF8575 blocks
<code>images/icons/expander.svg</code>	NEW	Category icon
<code>lib/NeoPixelStrip.py</code>	NEW	MicroPython driver for a WS2812 strip (change 6)
<code>js/lib/neopixel-source.js</code>	NEW	The same driver embedded for automatic installation on the robot
<code>js/blockly/blocks/neopixel.js</code>	NEW	NeoPixel block definitions
<code>js/blockly/generators/neopixel.js</code>	NEW	Python generators for the NeoPixel blocks
<code>images/icons/neopixel.svg</code>	NEW	Category icon
<code>js/app.js</code>	MODIFIED	Registers the new blocks and generators, customises the Variables flyout, post-processes generated code, installs drivers onto the robot
<code>js/blockly/toolbox.js</code>	MODIFIED	Adds repeat forever to the Loops category, plus the PCF8575 and NeoPixel categories
<code>js/blockly/theme.js</code>	MODIFIED	Block and category colours for the two new categories
<code>css/index.css</code>	MODIFIED	Icon masks for the two new categories
<code>js/ui/translations.js</code>	MODIFIED	English and Dutch strings for every new block
<code>serve.py</code>	NEW	No-cache local web server (change 12)
<code>js/ui/confirm-modal.js</code>	NEW	Reusable yes/no dialog (change 7)
<code>index.html</code>	MODIFIED	Confirmation dialog markup
<code>js/blockly/blocks/servo.js</code> <code>js/blockly/generators/servo.js</code>	MODIFIED	Four servos, clamping helper, centre, sweep and continuous speed (change 8)
<code>js/blockly/blocks/motors.js</code> <code>js/blockly/generators/motors.js</code>	MODIFIED	Translated labels, clamping helper, percent effort, speed control off, encoder counts (change 9)
<code>js/blockly/blocks/sensors.js</code>	MODIFIED	Gyro reset and heading, plus the gyro and line sensor

js/blockly/generators/sensors.js		dropdowns (changes 10 and 11)
js/blockly/blocks/drivetrain.js js/blockly/generators/drivetrain.js	MODIFIED	Wheel position dropdown (change 11)

Thirty-one files in total. The companion document `XRPBlocks-replication-guide.md` carries the full source of each one and rebuilds the whole installation from a clean clone of upstream.

What js/app.js now carries

The file has grown enough that line numbers would go stale; the replication guide supplies it whole. Its additions are:

Method or site	Purpose
Imports and registrations	The three new block families and their generators, the compatibility module, and the two embedded driver sources
<code>_customiseVariablesFlyout()</code>	Puts a shadow zero in the <i>set variable to socket</i> (change 3)
<code>patchMicroPythonImports()</code> call in <code>_generateCode()</code>	Strips CPython-only imports (change 4)
<code>_ensureLibraries()</code> , called before Run and Deploy	Copies each needed driver onto the robot once per board (changes 5 and 6)
<code>ConfirmModal.ask()</code> in <code>_handleDeploy()</code>	Asks before overwriting main.py (change 7)
<code>_pruneUnknownBlocks()</code>	Hides blocks the browser has not loaded, with a console warning and a toast (change 12)
<code>XRPLIB_OBJECTS</code>	Extended with <code>servo_three</code> and <code>servo_four</code> so the selective import covers them

After any change to these files, refresh the browser with **Ctrl+F5**. A normal refresh serves the cached JavaScript and the changes will appear to have had no effect.

14. Appendix: source of the new modules

js/blockly/blocks/loops.js

```
export function registerLoopBlocks() {
  // --- Repeat Forever ---
  Blockly.Blocks['xrp_forever'] = {
    init() {
      this.jsonInit({
        type: 'xrp_forever',
        message0: '%{BKY_XRP_FOREVER}',
        message1: '%1',
        args1: [
          { type: 'input_statement', name: 'D0' },
        ],
        previousStatement: null,
        // No nextStatement: nothing can follow a forever loop, which makes
        // the "this never ends" idea visible in the shape of the block.
        style: 'loop_blocks',
        tooltip: '%{BKY_XRP_FOREVER_TOOLTIP}',
        helpUrl: '',
      });
    },
  };
}
```

js/blockly/generators/loops.js

```
export function registerLoopGenerators(pythonModule) {
  const python = pythonModule.pythonGenerator;

  python.forBlock['xrp_forever'] = function (block, generator) {
    let branch = generator.statementToCode(block, 'D0');
    if (typeof generator.addLoopTrap === 'function') {
      branch = generator.addLoopTrap(branch, block);
    }
    if (!branch.trim()) {
      branch = generator.INDENT + 'pass\n';
    }
    if (!branch.endsWith('\n')) {
      branch += '\n';
    }
    return 'while True:\n' + branch;
  };
}
```

js/blockly/generators/micropython-compat.js

```
export function registerMicroPythonCompat(pythonModule) {
  const python = pythonModule.pythonGenerator;
  const Order = pythonModule.Order;

  python.forBlock['math_change'] = function (block, generator) {
    const delta = generator.valueToCode(block, 'DELTA', Order.ADDITIVE) || '0';
    const varName = generator.getVariableName(block.getFieldValue('VAR'));
    return `${varName} = ${varName} + ${delta}\n`;
  };
}

export function patchMicroPythonImports(code) {
  if (!code) return code;
  return code.replace(
    /^from numbers import Number$/gm,
    'Number = (int, float) # MicroPython has no "numbers" module'
  );
}
```

js/blockly/generators/expander.js (shared object helper)

```
const IMPORT_KEY = 'import_pcf8575';
const INSTANCE_KEY = 'pcf8575_instance';

function ensureExpander(generator, address) {
  generator.definitions_[IMPORT_KEY] = 'from PCF8575 import PCF8575, INPUT, OUTPUT';
  if (address) {
    generator.definitions_[INSTANCE_KEY] = `expander = PCF8575(address=${address})`;
  } else if (!generator.definitions_[INSTANCE_KEY]) {
    generator.definitions_[INSTANCE_KEY] = 'expander = PCF8575()';
  }
}
```

The NeoPixel generators use the same pattern with a `strip` object, so a program still works if the learner forgets the setup block; the setup block simply fixes the address, or the pin and pixel count.

js/app.js: installing a driver on the robot

```
async _ensureLibraries(code) {
  if (!code) return;
  const LIBRARIES = [
    { marker: 'PCF8575', filename: 'PCF8575.py', source: PCF8575_SOURCE },
    { marker: 'NeoPixelStrip', filename: 'NeoPixelStrip.py', source: NEOPIXEL_SOURCE },
  ];
  this._installedLibraries = this._installedLibraries || {};

  for (const lib of LIBRARIES) {
    if (!code.includes(lib.marker)) continue;
    if (this._installedLibraries[lib.filename]) continue;
    try {
      this.consolePanel?.appendSystem(`Installing ${lib.filename} on the robot...`);
      await this.serial.uploadFile(lib.filename, lib.source);
      this._installedLibraries[lib.filename] = true;
    } catch (err) {
      this.consolePanel?.appendError(`Could not install ${lib.filename}: ${err.message}`);
    }
  }
}
```

The full source of `lib/PCF8575.py` and `lib/NeoPixelStrip.py` is not reproduced here; both files are in the installation folder and are heavily commented, including the wiring rules.

Translation strings added to js/ui/translations.js

```
// English
XRP_FOREVER: 'repeat forever',
XRP_FOREVER_TOOLTIP: 'Repeat the blocks inside over and over, without stopping.'

// Dutch
XRP_FOREVER: 'herhaal oneindig',
XRP_FOREVER_TOOLTIP: 'Herhaal de blokken hierbinnen steeds opnieuw, zonder te stoppen.'
```

15. Verification status

Change	How it was verified
Module script blocked under <code>file://</code>	Confirmed against the installed <code>index.html</code> , line 23
Variables flyout inconsistency	Confirmed in the vendored Blockly build: the flyout adds a shadow to <i>change</i> but not to <i>set</i>
<code>numbers</code> import defect and its fix	Reproduced by running the XRPBlocks generator outside the browser against the failing program; before and after output inspected
All modified files	Parsed successfully; no syntax errors
PCF8575 driver	Run against a simulated I2C bus: byte order, shadow register, pins reserved as inputs never driven low, button reads leaving outputs undisturbed, address and pin range checks
NeoPixel driver	Run against a simulated strip: start-up blanking, brightness scaling and restoration, channel clamping, all three colour input formats, fill and clear, buffered mode writing nothing until show, shift wrapping both ways, a full lap being a no-op, ten distinct in-range rainbow hues that move with the offset, and every validation error firing
Blocks and toolbox	All thirteen categories build with no dangling block references; block definitions load into a real Blockly workspace, shadow blocks included
Generated programs	Generated from real block structures, then executed against the simulated hardware: both the PCF8575 and the NeoPixel programs ran cleanly with every value in range
Driver installation	Both embedded driver copies confirmed byte-identical to their <code>.py</code> files and confirmed to survive the raw-REPL escaping used to write them to the robot
Servo and motor blocks	A program using every block generated and then executed against simulated hardware: angle 250 clamps to 200, centre gives exactly 100, a 0.2 second sweep produces eleven monotonic steps, continuous speed -50 gives angle 50, motor effort 5 clamps to 1, -50 per cent becomes -0.5, and the encoder reset lands before the counts read
Confirmation dialog	Driven against the real markup in a DOM: Cancel focused on open, OK returns true, and the close button, Escape, the backdrop and a stray Enter all return false, with the dialog closing and the scroll lock released each time
Dropdown consolidation	All four dropdowns built and read back, all eight generated calls checked, and every retired block type confirmed to still have both a definition and a

	generator so saved work keeps loading
Cache failure and its fix	Reproduced by deleting block definitions before injecting a real workspace, which showed the cascade exactly as the browser reported it; with pruning in place the same test opens all thirteen categories
Replication	All 31 files extracted from the replication guide, copied into a clean clone of upstream <code>cd39757</code> , patched, and diffed: identical
Confirmed on hardware	The IDE end to end, deploy, the servo blocks, and the gyro blocks, on a SparkFun XRP Controller running MicroPython v1.25.0-preview.beta06
Still unconfirmed on hardware	The PCF8575 and NeoPixel drivers; whether the Qwiic 0 board pin names exist in this firmware (the driver falls back to GPIO 4 and 5, untested); and the NeoPixel electrical points about power, logic level and the servo header voltage

XRPBlocks: Local Modifications

Addendum: changes 13 to 18, covering the device library system, the OLED screen, a critical code-generation defect, and a full audit of every block

Prepared for: Edward Gentle, Subject Advisor CAT & IT, Capricorn South District, Limpopo Department of Education

Date: 19 September 2026

Installation: C:\Users\Gentle\Desktop\XRP\XRPBlocks-main\XRPBlocks-main

Upstream project: Stichting STEAMup / XRPBlocks (MIT licence), forked from commit `cd39757`

How this fits with Part One. The main document describes changes 1 to 12 and its summary in section 1 counts twelve. This addendum continues the same numbering with changes 13 to 18, so the installation now carries eighteen changes in total. Nothing in Part One has been withdrawn, but two of its statements are now out of date and are corrected here: the servo range in change 8, and the file manifest in section 13. Both are noted where they arise.

A1. Summary of changes 13 to 18

Six further changes were made. Two of them fix defects that stopped programs working, one is an architectural change that opens the IDE up to devices nobody has thought of yet, and three add capability. Section A7 lists every file touched since Part One.

#	Change	Problem it solves	Type
13	Dark mode	The pastel palette puts white text on very pale blocks. The yellow Board blocks measure 1.41:1 against white, far below the 4.5:1 that WCAG AA asks for normal text	Accessibility
14	Device library system	Adding a device meant editing five source files. Only the person who wrote the IDE could do it, so a teacher with a new sensor was stuck	Architecture
15	OLED screen library	No way to show a reading on the robot itself. The console needs a laptop on the end of a cable, which a robot driving across a classroom floor does not have	Feature
16	OLED drawing, compass arrow and the sketch designer	Text only. No way to draw, and no way to put a picture on the screen without writing byte arrays by hand	Feature
17	Functions were never generated	Every program using the Functions category called a function that was never defined. The robot stopped with NameError at the first call	Bug fix (critical)
18	Block audit: servo in degrees, blink repaired, one effort convention, clamping, units, toolbox order	A blink block that had never worked; a servo that was not in degrees; three different effort conventions in one session; unclamped drive effort; tooltips with no units	Bug fix and usability

Changes 14 to 16 are the important structural ones. The PCF8575 and NeoPixel categories from changes 5 and 6 were moved out of the IDE's source and into the new library format, which means the IDE no longer has any built-in knowledge of a specific add-on device.

A2. Change 13: Dark mode

Problem

The pastel theme draws white labels on very pale blocks. Measured against white text, the worst offender is the yellow used by the Board category, at roughly 1.41:1. WCAG 2.1 AA asks for 4.5:1 for normal text. In a bright classroom, or on a projector, several categories are hard to read.

Solution

A light and dark toggle in the toolbar, remembered in the browser so it survives a reload. Light mode is untouched, so nothing a teacher already recognises has moved. Dark mode keeps every category's hue, so blue is still Drive and green is still Sensors, but deepens each colour until white text on it passes 4.5:1.

The same rule is applied automatically to any colour a device library supplies. A library author gives one colour for light mode. If they do not also give a dark one, the IDE darkens theirs in small steps until the contrast test passes, so a library written by a teacher cannot accidentally produce an unreadable category.

A3. Change 14: The device library system

Problem

Changes 5 and 6 added the PCF8575 and NeoPixel categories, and each of them meant touching five files: a block definition file, a generator file, an embedded copy of the MicroPython driver, the toolbox, and the theme. That is a reasonable amount of work for somebody who wrote the IDE and an impossible amount for a teacher who has bought a new sensor.

Solution

A device library is now **one JSON file** that describes a device: its blocks, the Python each block generates, the category colour and icon, and the MicroPython driver to install on the robot. Nothing in the file is executed. It is data all the way down, which is what makes it safe to email a library to a colleague or fetch one from a web address.

```
{
  "manifestVersion": 1,
  "id": "pcf8575",
  "name": { "en": "PCF8575 I/O expander", "nl": "PCF8575 I/O-uitbreiding" },
  "category": { "key": "Expander", "colour": "#7E9BD8", "iconSvg": "<svg ...>" },
  "driver": { "filename": "PCF8575.py", "marker": "PCF8575", "source": "class PCF8575: ..." },
  "instance": { "name": "expander",
    "import": "from PCF8575 import PCF8575, INPUT, OUTPUT",
    "create": "PCF8575()",
    "createConfigured": "PCF8575(address={ADDR})" },
  "blocks": [ { "type": "xrp_pcf_led",
    "text": { "en": "PCF8575 LED on pin %1 %2" },
    "args": [ ... ],
    "code": "expander.led({PIN}, {STATE})\n" } ],
  "toolbox": [ { "block": "xrp_pcf_setup" }, { "gap": 16 } ]
}
```

How it behaves

- A **Library** button in the toolbar. Three ways in: the catalogue that ships with the IDE, a file from the computer, or a web address.
- PCF8575 and NeoPixel no longer appear by default, which gives a beginner a shorter toolbox. What a user adds is remembered and comes back on the next reload.
- Opening a saved project that uses blocks from a library the browser does not have puts that library back automatically, so work does not silently lose its blocks.
- A library cannot be removed while its blocks are still in the program. The dialog says so rather than quietly breaking the workspace.
- Drivers install onto the robot from the manifests, so a device added tomorrow installs its driver with no change to the IDE.
- A manifest that does not validate is rejected whole, with a list of what is wrong, rather than half-loading and breaking the palette.

The format is documented for somebody else to use in `devices/README.md`, which is written as a guide rather than as notes to self. Three libraries ship in `devices/`: the two moved categories and the OLED.

What the format deliberately cannot do. Because a manifest is data and never code, it cannot express a dropdown whose contents change with another field, a block that grows and shrinks, or validation logic beyond what the field types give. A device needing one of those still needs a hand-written block file. Everything the PCF8575 and NeoPixel categories did is expressible, which was the bar the format had to clear.

A4. Changes 15 and 16: The OLED screen library

The first library written from scratch to the published format rather than converted from existing code, and written without touching the IDE's own source. That was the test of whether the format holds. Change 16 then added nine blocks, a picture format and a complete drawing tool, again with no change to the IDE.

The hardware

DisplayModule DM-OLED096-636, a 0.96 inch 128 by 64 monochrome OLED. The controller is a Solomon Systech **SSD1315**, which takes the same commands as the far more common SSD1306, so the library covers both and works with most cheap 0.96 inch modules. It is an I2C device, so it goes on a Qwiic connector.

Detail	Value, and where it came from
Address	0x3C or 0x3D. The datasheet gives 0x78 and 0x7A, which is the 8-bit form including the read/write bit; MicroPython uses 7-bit addresses
Charge pump	Command 0x8D with 0x14 for 7.5 V, confirmed against the SSD1315 datasheet rather than assumed from the SSD1306
Text grid	Eight lines of sixteen characters, because the driver uses MicroPython's <code>framebuf</code> and its 8 by 8 font
Supply	DisplayModule lists the panel at 2.8 V. The usual breakout boards regulate 3.3 V down, but a bare module should be checked against its own datasheet before going on Qwiic

The blocks

Twenty-two, in five groups: setting up; text and numbers; arrows and pictures; drawing; display control; and three reporters.

Group	Blocks
Setting up	set up at address, clear
Text and numbers	print (console style, scrolls after eight lines), show text on line 1 to 8, show <code>label = value</code> on a line, write text at x and y
Arrows and pictures	arrow pointing N degrees, big text at 2x to 6x, show picture at x and y
Drawing	pixel, line, rectangle, circle, bar for a sensor reading
Display control	update now, update straight away or on command, brightness, normal or inverted, screen on and off
Reporters	is connected, screen width, screen height

The compass arrow

The arrow block takes an angle with 0 up, 90 right, 180 down and 270 left, which is exactly what the gyro heading block reports. Plugging the gyro block into the arrow block makes the needle swing as the robot turns, with no arithmetic for the learner to do. It is drawn with a shaft and a solid triangular head, clipped to the screen.

The sketch designer

`tools/oled-designer.html` is a mouse-driven sketch pad the size of the screen. It opens by double-clicking the file, or at `/tools/oled-designer.html` when the local server is running. Pencil, eraser, line, rectangle, circle, fill, invert, undo and redo, keyboard shortcuts, canvas sizes from the full screen down to an 8 by 8 sprite, previews at actual size and at double, and grid lines every eight pixels so the character cells and the display's page boundaries are visible.

Copy for XRP Blocks puts a string on the clipboard to paste into the **show picture** block. The same string can be pasted back into the designer to edit a drawing later. The format is one line of plain text:

```
XRPI1:128:64:AQD/AP8A/wD/AP8A/wD/AP8A ...
```

The decoded bytes begin with a mode byte, 0 for raw and 1 for run-length pairs, and the pixels are in the display's own memory layout so they go straight out to the panel. A mostly blank sketch shrinks to about forty characters; a noisy one falls back to raw at about fourteen hundred. Because it is text, it travels in a block, an email or a lesson file with nothing to encode by hand. The driver decodes it without `binascii`, so there is no dependency to worry about, and caches the decoded copy so redrawing inside a loop costs nothing after the first time.

A missing screen does not stop the robot. Unlike the PCF8575 driver, which raises if the chip does not answer, the OLED driver reports the problem on the console and then quietly does nothing. A robot should keep driving when its screen falls off. The consequence is that a program with no display attached looks exactly like a program that silently failed, so the Console tab is where the difference shows. The **OLED is connected** block lets a program check for itself.

A5. Change 17: Functions were never generated

This is the most serious defect found so far, and it had been present the whole time.

Problem

The IDE builds the Python by walking outwards from the **when program starts** block. That is deliberate: it means loose blocks left lying on the canvas are ignored rather than silently becoming part of the program. But **a function definition is a loose stack by nature**. Nothing connects it to the start block, so the walk never reached it and its body was never generated. The call came out; the definition did not.

```
oled.clear()
oled.set_auto_show(False)
for count in range(2):
    time.sleep(1)
    UP()          <-- called, but there is no def UP() anywhere in the program
oled.show()
```

On the robot that is `NameError: name 'UP' isn't defined` at the first call, so the program stops before doing anything. Every program using the Functions category was affected, for any device, not only the OLED ones. It presented as "the program runs and nothing happens", because the traceback appears only in the Console tab, which a learner has no particular reason to open.

Solution

A new step in `js/app.js` generates every function definition on the canvas before the start stack is walked. Their generators put the code into Blockly's definitions table, which is emitted at the top of the program, ahead of everything that calls them.

```
// A function definition is its own stack on the canvas: nothing reaches it
// from the start block, so walking out from the start block alone never
// generates its body.
this._generateFunctionDefinitions();

code = this._pythonGenerator.blockToCode(startBlock);
code = this._pythonGenerator.finish(code);
```

Both kinds of function are covered, those that return a value and those that do not.

Worth reporting upstream, alongside the `numbers` import defect from change 4. If the upstream project generates from the start block in the same way, it has the same defect.

A6. Change 18: The block audit

Every block in the IDE was read against the XRPLib *source*, not only its documentation, and then exercised in a real browser. Four problems came out of it.

The blink block had never worked

It generated `board.led_blink(3, 0.5)`. XRPLib's `led_blink` takes **one** argument, a frequency in Hz, and starts a background timer. Two arguments raise `TypeError` and the program stops on that line. The block now generates a proper count-and-delay loop, so it does what its label promises and the program waits while it blinks.

The servo was never in degrees

This corrects change 8 in Part One, which described the range as 0 to 200 with 100 at the centre. That was accurate but it was never really an angle. XRPLib's `set_angle(d)` converts straight to a pulse width, $d * 10 + 500$ microseconds, so its 0 to 200 is the full 500 to 2500 microsecond travel of a hobby servo. A student with a protractor had no way to make sense of it.

The blocks now take **0 to 180 degrees**, mapped across the full travel by one generated helper using $\text{degrees} * 10 / 9$.

The block says	XRPLib receives	Pulse width
0 degrees	5.6	556 microseconds
90 degrees	100	1500 microseconds, the centre
180 degrees	194.4	2444 microseconds

Five degrees are held back at each end, so a servo is never driven into its own mechanical stop, where it buzzes, draws current and gets hot. The two constants that do this are written at the top of the generated helper, so a class with a servo that can take the full sweep can widen them by hand and see the effect immediately in the Python panel.

```
def xrp_servo_degrees(servo, degrees):
    SAFE_LOW = 5
    SAFE_HIGH = 175
    if degrees < SAFE_LOW:
        degrees = SAFE_LOW
    elif degrees > SAFE_HIGH:
        degrees = SAFE_HIGH
    servo.set_angle(degrees * 10 / 9)
```

Centre is now 90 rather than 100, sweep is in degrees and defaults to 0 through 180, and continuous rotation speed stays at -100 to 100 percent, mapped around the 90 degree centre. The change was made in place rather than by adding a second block, so there is one angle block rather than two. A saved project that said 150 meant three quarters of travel and now means 150 degrees, which is a different position. Anything already deployed to a robot as `main.py` is unaffected.

Effort had three conventions at once

A student could meet "drive straight with 50% effort", "set drive effort 0.5", and a Motors category offering both a 0 to 1 block and a percent block, all in one lesson. Effort is now percent everywhere a student can see, converted and clamped by one helper.

```
def xrp_effort(percent, lowest=-1):
    effort = percent / 100
    if effort < lowest:
        effort = lowest
    elif effort > 1:
        effort = 1
    return effort
```

`lowest=0` is used for the maximum effort of a measured drive, where a negative value makes no sense. The 0 to 1 motor block stays registered so saved projects still load, but is no longer offered in the toolbox.

Drive effort and arcade were unclamped

The Motors and Servo categories already clamped. `drivetrain.set_effort` and `drivetrain.arcade` did not, and XRPLib does not either, so a slip of the keyboard went straight to the motor driver. Both now go through the helper above.

Units were missing where they matter most

`drivetrain.set_speed` is centimetres per second but `motor.set_speed` is revolutions per minute, and both blocks simply said "speed". Reflectance returns a decimal from 0 to 1, not a percentage. Every unit-bearing tooltip now names its unit and its range, in English and in Dutch.

Every category reordered

Each category now runs from what a beginner needs down to the advanced, with separators marking each group and a comment in the source saying what each group is for.

Category	Order, from the top
Drive	straight, turn, stop the same two with a power setting set effort, arcade, set speed, none of which stop by themselves the encoders
Motors	effort in percent, brake, coast speed control the encoder
Servo	set an angle, centre sweep continuous rotation, free
Sensors	distance, line, button the gyro, which needs zeroing first
Board	the green LED wait and print the RGB LED

A7. File manifest since Part One

This supersedes section 13 of Part One for everything added or changed since. Part One's manifest remains correct for changes 1 to 12, except for the six files now retired, listed at the end.

Added

File	What it is
devices/index.json	Catalogue of the libraries that ship with the IDE
devices/pcf8575.json	The PCF8575 category, moved out of the source
devices/neopixel.json	The NeoPixel category, moved out of the source
devices/ssd1315-oled.json	The OLED library, 22 blocks
devices/README.md	The manifest format, written for somebody else to use
js/devices/manifest-compiler.js	Manifest to Blockly blocks, Python generators and a toolbox category, plus the validator
js/devices/library-manager.js	Fetching, validating, registering, theme colours, driver list, persistence
js/ui/library-modal.js	The Library dialog
lib/SSD1315.py	The OLED driver, embedded byte for byte in the manifest
tools/oled-designer.html	The mouse-driven sketch pad

Changed

File	Why
index.html	Library button and its dialog
css/index.css	Dark mode, and the Library dialog
js/app.js	Library loading, theme, driver installation from manifests, and the function generation fix
js/blockly/theme.js	Dark palette, and styles contributed by libraries
js/blockly/toolbox.js	Library categories, and every category reordered
js/blockly/generators/servo.js	Degrees
js/blockly/generators/board.js	The blink fix
js/blockly/generators/drivetrain.js	Effort in percent, and clamping
js/ui/toolbar.js	Library button
js/ui/translations.js	Dialog strings, and units in every tooltip that lacked one

Retired, safe to delete

Nothing imports these any longer. They were the PCF8575 and NeoPixel categories before those moved into the library format.

js/blockly/blocks/expander.js	js/blockly/generators/expander.js
js/blockly/blocks/neopixel.js	js/blockly/generators/neopixel.js
js/lib/pcf8575-source.js	js/lib/neopixel-source.js

`lib/PCF8575.py` and `lib/NeoPixelStrip.py` stay. They are the canonical driver source that the manifests were built from.

A8. Verification status

Eight automated harnesses, 913 checks, all passing at the time of writing. They run the real code: five of them drive the actual IDE in a headless browser rather than testing a copy of the logic.

Checks	Harness	What it proves
353	Manifests against the old code	The PCF8575 and NeoPixel manifests reproduce the hand-written blocks exactly: labels, shapes, arguments, dropdown options, the Python generated for every dropdown combination with sockets full and empty, toolbox shadows, driver source byte for byte, and eleven kinds of bad manifest

		being rejected
44	Library manager	Adding, removing, refusing removal while blocks are in use, clash detection, persistence across a reload, falling back to the stored copy when a fetch fails, and the contrast rule
39	The IDE end to end	The dialog, adding libraries, categories appearing, flyouts rendering, generated Python, reload, dark mode, and the functions regression test from change 17
194	Every block validated	Every one of the 48 registered blocks built in the real IDE and its Python checked against the XRPLib API. Every dropdown is exercised through every option, all four motors, all four servos, all four gyro readings. The harness asks the IDE which blocks it has registered and fails if any is not covered, so a block added later cannot slip through untested
169	The OLED driver	Run against stand-ins for <code>framebuf</code> and <code>machine</code> : the initialisation sequence, the 1025 byte frame, line arithmetic, number formatting, the console scroll, brightness, both Qwiic buses, a missing display, a display that dies mid-run, the drawing primitives, arrow direction and symmetry, big-text centring, picture decoding, and six kinds of malformed picture
32	The designer page	Real mouse drags, every tool, undo and redo, shape preview not committing until the mouse is released, fill and invert and clear, keyboard shortcuts, canvas resizing, and loading a string back
44	The cross-language check	The designer draws fourteen test pictures and the driver's Python decoder must return exactly the same pixels for every one, in both compression modes. Two separate implementations of one format is where a silent drift would hurt most
38	The OLED in the IDE	The validator, adding from a file, all 22 blocks rendering, a compass program with the gyro plugged into the arrow, the picture string surviving a text block unaltered, reload, dark colour contrast, and removal

Confirmed on hardware

The IDE end to end, deploy, the servo blocks and the gyro blocks, on a SparkFun XRP Controller running MicroPython v1.25.0-preview.beta06.

The OLED library, 19 September 2026: works as expected. That settles three questions this document had left open. This firmware does have `framebuf`, so the driver's import guard never fires. The module answers at 0x3C, the address the setup block defaults to. And the driver install path over the raw REPL works for a driver supplied by a device library rather than built into the IDE, which was the one part of change 14 that could only be proved on hardware.

Still to confirm on hardware

Shorter than it was. The OLED items have gone; what remains is either untested hardware or a question only a robot on a floor can answer.

Item	What to look for
The PCF8575 and NeoPixel drivers	Unchanged in behaviour by the move to the library format, but still never run on a robot
The servo end stops	Whether 0 and 180 degrees move freely. If they do, widen the two constants in the helper; if the servo buzzes at the ends, narrow them
Which way a positive arcade turn steers	The source computes left as straight minus turn and right as straight plus turn, so a positive turn runs the right wheel faster. Two summaries of that code disagreed about the resulting direction, so the tooltip describes the wheels rather than naming left or right. Five minutes on the floor settles it, and then the tooltip can say it plainly
Redraw speed	An arrow redrawn in a tight loop sends 1025 bytes over I2C each pass. If it drags, set updating to "only when I say update" and use one update block per pass
Board pin names	Whether <code>I2C_SDA_0</code> and <code>I2C_SCL_0</code> exist in this firmware. All three drivers fall back to raw GPIO numbers, untested

A note on the companion document. XRPBlocks-replication-guide.md still reflects the installation as it stood at change 13 and has not been regenerated for changes 14 to 18. The device libraries, the OLED driver and the designer page are each self-contained files in the installation folder, so they can be copied across directly; the changed IDE files are the ten listed in section A7.